

Disjoint Parallel Programs (continue)

How to recognize a disjoint parallel program?

- For statement S , we define that:
 - $vars(S)$ = the set of variables in S . (We either read or write these variables in S)
 - $change(S)$ = the set of variables appears on the left-hand side of assignments in S . (We write these variables in S)

We say thread S_i **interferes** with S_j if $change(S_i) \cap vars(S_j) \neq \emptyset$.

We say threads S_i and S_j are **disjoint**, if $change(S_i) \cap vars(S_j) = change(S_j) \cap vars(S_i) = \emptyset$.

- If for $0 < i \neq j \leq n$, S_i and S_j are disjoint, then we say threads $S_1, S_2, \dots, S_n$ are **pairwise disjoint**, and we say $[S_1 \parallel S_2 \parallel \dots \parallel S_n]$ is a **disjoint parallel composition**. We also say $[S_1 \parallel S_2 \parallel \dots \parallel S_n]$ is a **disjoint parallel program**.

1. Determine whether the following threads are disjoint.

a) $S_1 \equiv a := a + x$ and $S_2 \equiv y := y + x$

Yes. $vars(S_1) = \{a, x\}$ and $change(S_2) = \{y\}$, then $vars(S_1) \cap change(S_2) = \emptyset$; $changes(S_1) = \{a\}$ and $vars(S_2) = \{y, x\}$, then $vars(S_2) \cap change(S_1) = \emptyset$.

b) $S_1 \equiv a := x$ and $S_2 \equiv x := c$

No, $vars(S_1) = \{a, x\}$ and $change(S_2) = \{x\}$, then $change(S_2) \cap vars(S_1) \neq \emptyset$; thus S_2 interferes with S_1 .

c) $S_1 \equiv x := a + 1$ and $S_2 \equiv x := b + 1$

No, both S_1 and S_2 write x , so they interfere with each other.

2. Is the program $S \equiv [S_1 \parallel S_2 \parallel S_3]$ a disjoint parallel program? Here, we have

$S_1 \equiv a := v; v := v + c$

$S_2 \equiv \text{if } b > 0 \text{ then } b := b * 2 \text{ else } c := c * 2 \text{ fi}$

$S_3 \equiv \text{while } d \geq 0 \text{ do } d := d \div 2 - c \text{ od}$

- We can come up with the following table:

i	j	$vars(S_i)$	$changes(S_j)$	S_j interferes with S_i ?
1	2	a, v, c	b, c	Yes
2	1	b, c	a, v	No
1	3	a, v, c	d	No
3	1	d, c	a, v	No
2	3	b, c	d	No
3	2	d, c	b, c	Yes

To sum up, S_2 interferes with both S_1 and S_3 , thus the program S is **not disjoint parallel**.

- In the above example, we can see that S_2 only interferes with S_1, S_3 if we execute the false branch: If $b > 0$ at the runtime, these threads will not interfere with each other.

- In fact, this “disjointedness test” we use here is static and can be too strict, it can over-estimate the interference:
 - If our test shows that each pair of threads are disjoint, then the program is definitely a disjoint parallel program.
 - If our test shows that each thread can interfere with other threads, then it only means that this thread **might** interfere with other threads while executing.

Inference Rules for Disjoint Parallel Programs

Due to the Unique Result of Disjoint Parallel Program Theorem, we can come up with the following rule:

- **Sequentialization Rule**

If $S_1, S_2, \dots, S_n$ are pairwise disjoint:

1. $\{p\} S_1; S_2; \dots; S_n \{q\}$
2. $\{p\} [S_1 \parallel S_2 \parallel \dots \parallel S_n] \{q\}$ sequentialization 1

- We call the **sequentialization** of the parallel statement $[S_1 \parallel S_2 \parallel \dots \parallel S_n]$ (not necessarily disjoint parallel) is the sequence statement $S_1; S_2; \dots; S_n$. The **sequentialized execution** of the parallel statement is the execution of its sequentialization: We evaluate S_1 completely, then S_2 completely, and so on.
- In disjoint parallel programs, the execution order of threads do not matter. Thus, while using the sequentialization rule in proof, we can write in the following way as well:

1. $\{p\} S_2; S_1 \{q\}$
2. $\{p\} [S_1 \parallel S_2] \{q\}$ sequentialization 1

3. Give a formal proof of $\{T\} [a := x + 1 \parallel b := x + 2] \{a + 1 = b\}$.

- We can show threads are pairwise disjoint to apply the sequentialization rule. We can give the following Hilbert style proof.

1. $\{a + 1 = x + 2\} b := x + 2 \{a + 1 = b\}$ backward assignment
2. $\{x + 1 + 1 = x + 2\} a := x + 1 \{a + 1 = x + 2\}$ backward assignment
3. $\{x + 1 + 1 = x + 2\} a := x + 1; b := x + 2 \{a + 1 = b\}$ sequence 2,1
4. $T \Rightarrow x + 1 + 1 = x + 2$ predicate logic
5. $\{T\} a := x + 1; b := x + 2 \{a + 1 = b\}$ strengthen precondition 4,3

#

i	j	$vars(S_i)$	$changes(S_j)$	S_j interferes with S_i ?
1	2	a, x	b	No
2	1	b, x	a	No

6. $\{T\} [a := x + 1 \parallel b := x + 2] \{a + 1 = b\}$ sequentialization 5

In the above example, we can see that Sequentialization rule is not easy to use when there are many threads, we need to calculate a lot of pre- or post- conditions to prove sequentialization.

Let's come up with some rules of inference that are easy to use. Someone suggested that, using the fact that disjoint threads can write at the same time without affecting variables in other threads, we come up with the parallelism rule.

- **Parallelism Rule (v 1.0)?**

If $S_1, S_2, \dots, S_n$ are pairwise disjoint:

1 $\{p_1\} S_1 \{q_1\}$

2 $\{p_2\} S_2 \{q_2\}$

...

n $\{p_n\} S_n \{q_n\}$

n + 1 $\{p_1 \wedge p_2 \wedge \dots \wedge p_n\} [S_1 \parallel S_2 \parallel \dots \parallel S_n] \{q_1 \wedge q_2 \wedge \dots \wedge q_n\}$ parallelism 1, 2, ..., n

- In the format of proof outline, we write: $\{p_1 \wedge p_2 \wedge \dots \wedge p_n\} [\{p_1\} S_1 \{q_1\} \parallel \{p_2\} S_2 \{q_2\} \parallel \dots \parallel \{p_n\} S_n \{q_n\}] \{q_1 \wedge q_2 \wedge \dots \wedge q_n\}$

4. Create a full proof outline for $\{x = y = z = c\} [x := x^2 \parallel y := y^2 \parallel z := (z - d) * (z + d)] \{x = y = z + d^2\}$, where c, d are named constants.

First, we need to prove that all threads are pairwise disjoint.

i	j	$vars(S_i)$	$changes(S_j)$	S_j interferes with S_i ?
1	2	x	y	No
2	1	y	x	No
1	3	x	z	No
3	1	z	x	No
2	3	y	z	No
3	2	z	y	No

Using parallelism rule, we can come up with the following full proof outline:

$\{x = y = z = c\} \{x = c \wedge y = c \wedge z = c\}$
 $[\{x = c\} x := x^2 \{x_0 = c \wedge x = x_0^2\} \{x = c^2\}$
 $\parallel \{y = c\} y := y^2 \{y_0 = c \wedge y = y_0^2\} \{y = c^2\}$
 $\parallel \{z = c\} z := (z - d) * (z + d) \{z_0 = c \wedge z = (z_0 - d) * (z_0 + d)\} \{z = c^2 - d^2\}]$
 $\{x = c^2 \wedge y = c^2 \wedge z = c^2 - d^2\}$
 $\{x = y = z + d^2\}$

- The parallelism rule (v 1.0) is not correct, it can fail in some disjoint parallel programs.
5. For each of the following proof outlines, point out what is wrong in the proof and why parallelism rule (v 1.0) does not work.
 - a) $\{x = 0\} [\{x = 0\} x := 1 \{x = 1\} \parallel \{x = 0\} y := 0 \{x = y = 0\}] \{x = 1 \wedge x = y = 0\}$
 - The **postcondition** is wrong: $x = 1 \wedge x = y = 0$ is false, but this is not the case if we analyze the program.
 - The assignment $x := 1$ in the first thread can **make the precondition $x = 0$ of the second thread not true anymore**.
 - The assignment $x := 1$ also **interferes with the postcondition $x = y = 0$ of the second thread, because the x in this postcondition represents the x that's before assignment $x := 1$.**
 - b) $\{x \geq y \wedge y = z\} [\{x \geq y\} x := x + 1 \{x > y\} \parallel \{y = z\} y := y * 2; z := z * 2 \{y = z\}] \{x > y \wedge y = z\}$
 - The **postcondition** is wrong: we don't always have $x \geq y$ in the postcondition. For example, if we execute the program in a state where x, y, z all evaluated to 2, then we will end up with x updated to 3, and y, z both updated to 4.

- The assignment $y := y * 2$ in the second thread can make the precondition $x \geq y$ of the first thread not true anymore.
- The assignment $y := y * 2$ also interferes with the postcondition $x > y$ of the second thread, because the y in this postcondition represents the y that's before assignment $y := y * 2$.
- We have the following observation. So that parallelism rule can work in a disjoint parallel program, we need more: "a thread doesn't change variables appear in the pre- and post- conditions of other threads".
- For a set of predicates $p_1, p_2, \dots, p_n$, we define $free(p_1, \dots, p_n)$ = the set of variables that are free (have free occurrences) in $p_1, p_2, \dots, p_n$.
- Given triples $\{p_1\} S_1 \{q_1\}$ and $\{p_2\} S_2 \{q_2\}$, we say S_1 interferes with the conditions of S_2 if $change(S_1) \cap free(p_2, q_2) \neq \emptyset$. And we say S_1 and S_2 have disjoint conditions if $change(S_1) \cap free(p_2, q_2) = \emptyset$ and $change(S_2) \cap free(p_1, q_1) = \emptyset$.

Here, we give the real parallelism rule.

- **Parallelism Rule**

If $S_1, S_2, \dots, S_n$ are pairwise disjoint, and $S_1, S_2, \dots, S_n$ have disjoint conditions:

- $$\begin{array}{ll}
 1 & \{p_1\} S_1 \{q_1\} \\
 2 & \{p_2\} S_2 \{q_2\} \\
 & \dots \\
 n & \{p_n\} S_n \{q_n\} \\
 n+1 & \{p_1 \wedge p_2 \wedge \dots \wedge p_n\} [S_1 \parallel S_2 \parallel \dots \parallel S_n] \{q_1 \wedge q_2 \wedge \dots \wedge q_n\} \quad \text{parallelism } 1, 2, \dots, n
 \end{array}$$

6. Are the following threads pairwise disjoint? Do they have disjoint conditions?

- $$\begin{array}{l}
 \{p_1\} S_1 \{q_1\} \equiv \{x \geq y\} x := x + 1 \{x > y\} \\
 \{p_2\} S_2 \{q_2\} \equiv \{y = z\} y := y * 2; z := z * 2 \{y = z\}
 \end{array}$$

We can come up with the following table:

i	j	$change(S_i)$	$vars(S_j)$	$free(p_j, q_j)$	$S_i \text{ intf } S_j$	$S_i \text{ intf cond}_j$
1	2	x	y, z	y, z	No	No
2	1	y, z	x	x, y	No	Yes

So, S_1 and S_2 are disjoint, but they don't have disjoint conditions. That's why we cannot guarantee that we can use the parallelism rule to prove $\{p_1 \wedge p_2\} [S_1 \parallel S_2] \{q_1 \wedge q_2\}$.

Removing Interference of Conditions for Disjoint Parallel Programs

What if we want to find a way to find a way to prove disjoint parallel programs using the parallelism rule? We want to find a way to remove the interference of conditions among disjoint threads. This can be done by **aging all the free variables in the precondition**.

7. Find some q , and use the parallelism rule to prove the following parallel program.

$$\underbrace{\{x = 0\}}_{\text{pre}} [\underbrace{\{x = 0\}}_{\text{pre}} \underbrace{x := 1}_{\text{post}} \underbrace{\{x = 1\}}_{\text{post}} \parallel \underbrace{\{x = 0\}}_{\text{pre}} \underbrace{y := 0}_{\text{post}} \underbrace{\{x = y = 0\}}_{\text{post}}] \{q\}$$

- S_1 interferes the conditions of S_2
 - The assignment $x := 1$ can make the precondition $x = 0$ not true anymore.
 - The assignment $x := 1$ also interferes with the postcondition $x = y = 0$, because the x in this postcondition represents the x that's before assignment $x := 1$.
- To avoid this interference, we can age all the x in preconditions.

- c) For the first thread: $\{x_0 = 0 \wedge x = x_0\} x := 1 \{x_0 = 0 \wedge x = 1\} \{x = 1\}$
d) For the second thread: $\{x_0 = 0\} y := 0 \{x_0 = 0 \wedge y = 0\}$

Thus, we can come up with the following full proof outline that uses parallelism rule:

$$\begin{aligned} & \{x_0 = 0 \wedge x = x_0\} \\ & [\{x_0 = 0 \wedge x = x_0\} x := 1 \{x_0 = 0 \wedge x = 1\} \{x = 1\} \parallel \\ & \{x_0 = 0\} y := 0 \{x_0 = 0 \wedge y = 0\}] \\ & \{x_0 = 0 \wedge x = 1 \wedge y = 0\} \end{aligned}$$

8. Find some q , and use the parallelism rule to prove the following parallel program.

$$\{x \geq y \wedge y = z\} [\{x \geq y\} x := x + 1 \{x > y\} \parallel \{y = z\} y := y * 2; z := z * 2 \{y = z\}] \{q\}$$

- We know that the assignment $y := y * 2$ interferes with the pre- and post- conditions of the first thread; and we can age y to avoid this interference. However, **aging all the free variables in preconditions** is a simple trick.

For every free variable x in the precondition of a thread:

- If other threads write it, then aging x can remove interference in conditions.
- If the current thread writes it, then aging x is needed for forward assignment.
- If no thread writes x , then x is not updated anywhere and aging it is safe.

- For the first thread: $\{x_0 \geq y_0 \wedge x = x_0\} x := x + 1 \{x_0 \geq y_0 \wedge x = x_0 + 1\} \{x > y_0\}$.
- For the second thread: $\{y_0 = z_0 \wedge y = y_0 \wedge z = z_0\} y := y * 2; \{y_0 = z_0 \wedge y = y_0 * 2 \wedge z = z_0\} z := z * 2 \{y_0 = z_0 \wedge y = y_0 * 2 \wedge z = z_0 * 2\} \{y = z\}$.

Thus, we can come up with the following full proof outline that uses parallelism rule:

$$\begin{aligned} & \{x_0 \geq y_0 \wedge y_0 = z_0 \wedge x = x_0 \wedge y = y_0 \wedge z = z_0\} \\ & [\{x_0 \geq y_0 \wedge x = x_0\} x := x + 1 \{x_0 \geq y_0 \wedge x = x_0 + 1\} \{x > y_0\} \parallel \\ & \{y_0 = z_0 \wedge y = y_0 \wedge z = z_0\} y := y * 2; \{y_0 = z_0 \wedge y = y_0 * 2 \wedge z = z_0\} \\ & \quad z := z * 2 \{y_0 = z_0 \wedge y = y_0 * 2 \wedge z = z_0 * 2\} \{y = z\}] \\ & \{x > y_0 \wedge y = z\} \end{aligned}$$